Expanded upon by Stefan Scott

Sound – 01 - Making Music

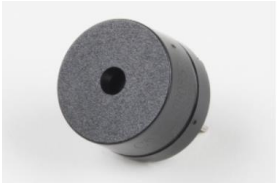
Purpose: To wire and code the Arduino to play “Happy Birthday.”

Materials: Computer, Arduino Uno Board, USB cable, Breadboard, 2 coloured LED’s, 200 - 330 Ω Resistor, 2 Jumper Wires

New Components:

Piezo Buzzer

The buzzer uses a small magnetic coil to vibrate a metal disc inside a plastic housing. By pulsing electricity through the coil at different rates, different frequencies (pitches) of sound can be produced. Attaching a potentiometer to the output allows you to limit the amount of current moving through the buzzer and lower its volume. It works very similar to a speaker.







Please note on the underside of the buzzer that there is a positive and negative leg. All of the circuits in Project 2 make use of a potentiometer as a rudimentary volume knob. Notice that only two of the potentiometer’s legs are used in these circuits. In these instances, the potentiometer is acting as a variable resistor, limiting the amount of current flowing to the speaker and thus affecting the volume as you turn the knob.

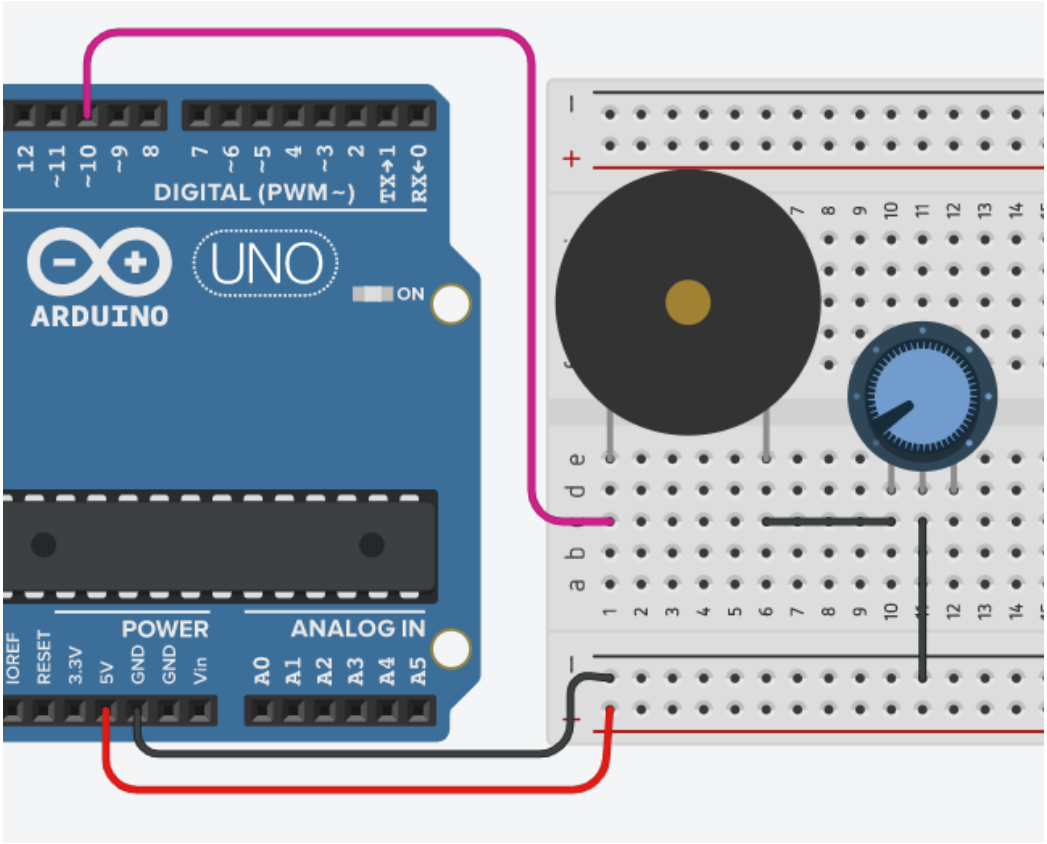
With funding from Canada

<SaskCode>

SIEC Saskatoon Industry Education Council

Procedure:

1. Wire the project using the following schematic.



2. You can create your code in the TinkerCAD Editor then paste it into the Arduino IDE to upload the code to your Arduino.

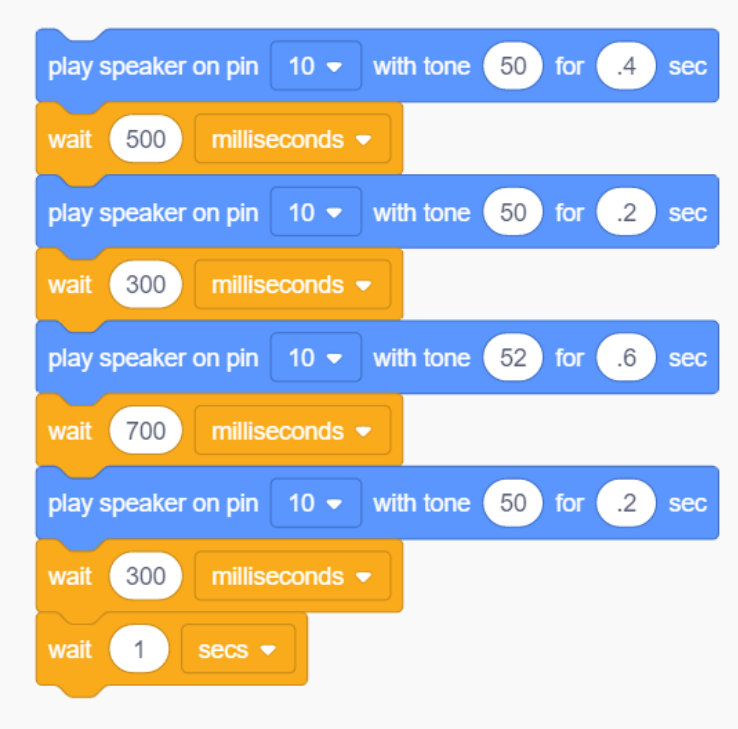
3. Once in TinkerCAD, use the following blocks and put them into the editor to get your tune to play on a speaker. You can also control the volume with the potentiometer (but only over a narrow range of the potentiometer).

4. Below are the blocks that you will need. Remember you can set your coding environment to **Blocks and Text**; try to see the connection between the block syntax and the text syntax.

   A. Start by making the first chorus. You will need to select a tone pin, a frequency (sound that is played), and the length of the note. Below is our tone pin 10, the frequencies needed and the duration of the note which is in milliseconds. There are 1000 milliseconds in a second.

      Notice that there is a delay after each **playSpeaker** block; this delay allows

the sound to play its full length before a new sound instruction is given. In this example below, the *delay is slightly longer than the tone length;* this allows for a small break in between notes. If you don't need a break, the delay can match the play length exactly.



B. At this point, switch to the **Blocks and Text** view, and take notice of the code there (in particular, the comments that were generated):

```
8    tone(10, 294, 400); // play tone 50 (D4 = 294 Hz)
9    delay(500); // Wait for 500 millisecond(s)
```

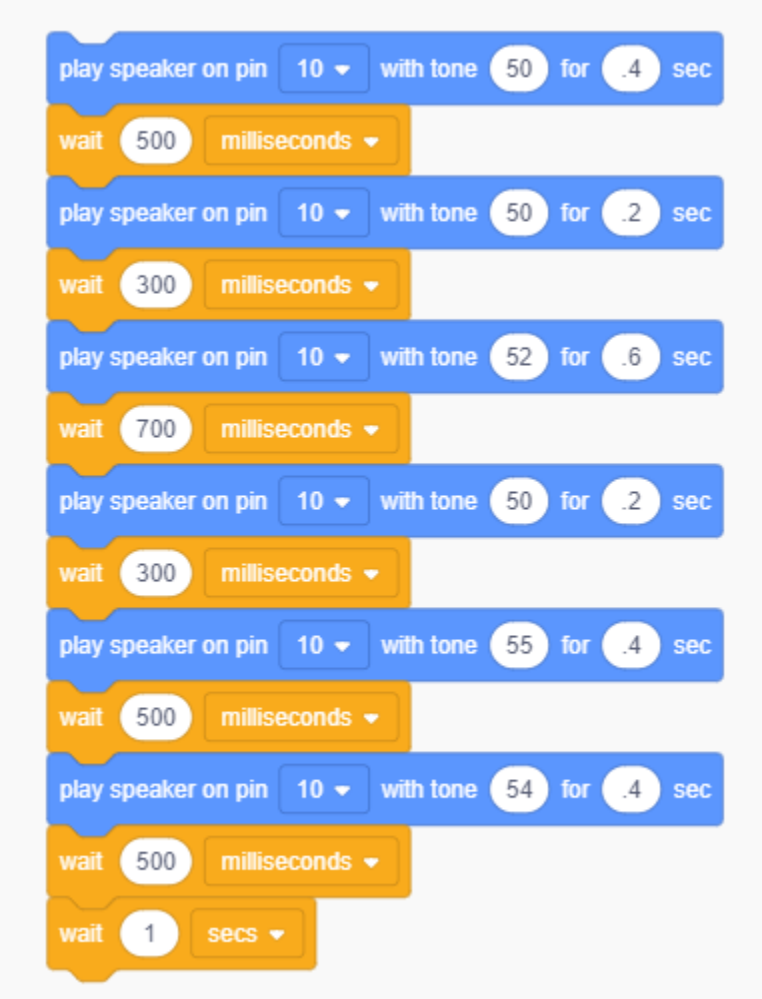
Something interesting is the **tone 50** gets translated. In the comment, we can see that tone number 50 is a **D** (d4, specifically), which is 294 Hz.

In fact, in code the number 50 isn't used at all! TinkerCAD uses this tone numbering system to make it easier to write songs in code; ultimately the Arduino code just wants to know the frequency of each tone. So if you were editing the **text code** down the line, remember that the second value in the **tone()** function is the *frequency of the note to be played in Hertz.*

C. Let's go a little further with our song: Some blocks will be supplied below to create yourself. **Remember:** you can use the **duplicate** command to save time when some code you want is similar to something you already have:



The block right-clicked on and all blocks connected below it are duplicated






```
void setup()
{
  pinMode(10, OUTPUT);
}

void loop()
{
  tone(10, 294, 400); // play tone 50 (D4 = 294 Hz)
  delay(500); // Wait for 500 millisecond(s)
  tone(10, 294, 200); // play tone 50 (D4 = 294 Hz)
  delay(300); // Wait for 300 millisecond(s)
  tone(10, 330, 600); // play tone 52 (E4 = 330 Hz)
  delay(700); // Wait for 700 millisecond(s)
  tone(10, 294, 200); // play tone 50 (D4 = 294 Hz)
  delay(300); // Wait for 300 millisecond(s)
  tone(10, 392, 400); // play tone 55 (G4 = 392 Hz)
  delay(500); // Wait for 500 millisecond(s)
  tone(10, 370, 400); // play tone 54 (F#4 = 370 Hz)
  delay(500); // Wait for 500 millisecond(s)
  delay(1000); // Wait for 1000 millisecond(s)

  tone(10, 294, 400); // play tone 50 (D4 = 294 Hz)
  delay(500); // Wait for 500 millisecond(s)
  tone(10, 294, 200); // play tone 50 (D4 = 294 Hz)
  delay(300); // Wait for 300 millisecond(s)
  tone(10, 330, 600); // play tone 52 (E4 = 330 Hz)
  delay(700); // Wait for 700 millisecond(s)
  tone(10, 294, 200); // play tone 50 (D4 = 294 Hz)
  delay(300); // Wait for 300 millisecond(s)
  tone(10, 440, 400); // play tone 57 (A4 = 440 Hz)
  delay(500); // Wait for 500 millisecond(s)
  tone(10, 392, 400); // play tone 55 (G4 = 392 Hz)
  delay(500); // Wait for 500 millisecond(s)
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Challenge One – Finish the Song

- There is a bit of the “Happy Birthday” tune missing. Using blocks similar to above to construct the missing portion. Look for the patterns in the existing code (timing, for instance) to help your intuition for the correct tone and length values.

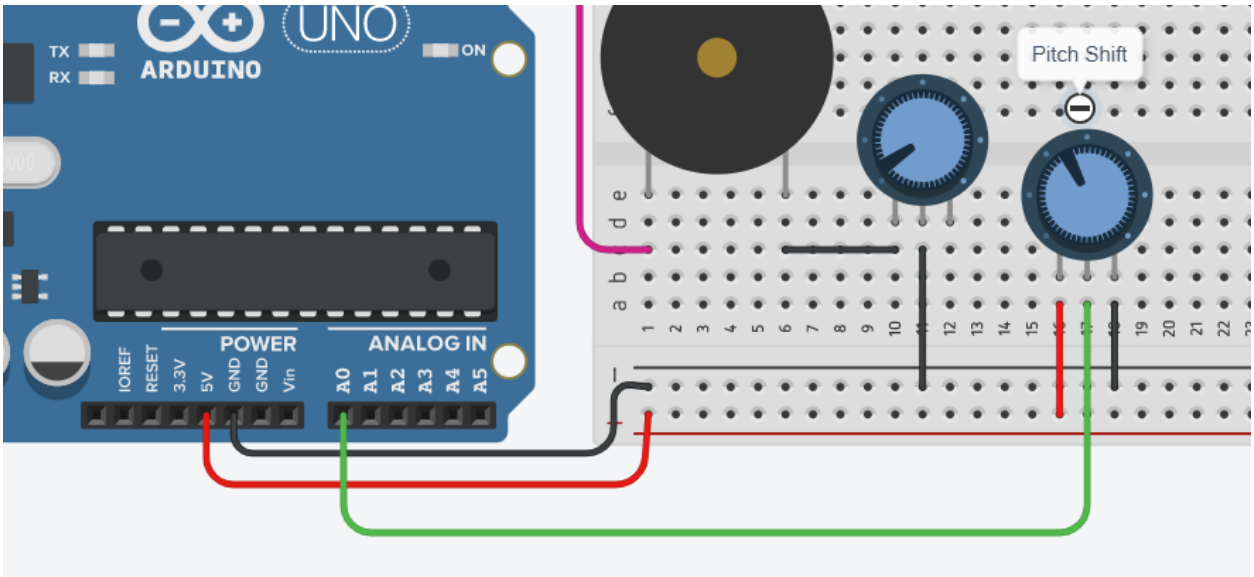
- Add one last delay of 5 seconds at the end of the song so that there is a break before the program repeats and the song starts again.

At this point, make and test the physical circuit. Once it is working successfully, move on to Challenge Two back in the simulator.

Challenge Two – Pitch Control

Presently all the tone values are *hardcoded;* that is; they can't change while the program is running. We could add a **potentiometer** to our circuit, and use its position to shift the pitch of each note

- Start with adding a potentiometer to our circuit, with the data wire connected to A0

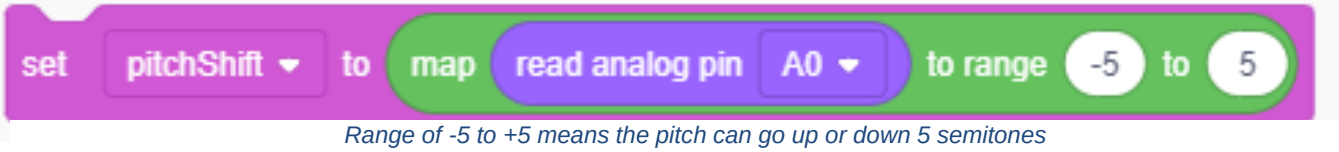


- Next, create a variable called **pitchShift** that we'll use to store *how much to change the pitch by.* A few things to consider:

  - the tone values should land in a range of (15-75). The piezo can make higher sounds, but it may not be pleasant to listen to!

  - analogRead() gives us a value from 0-1023, so likely a value that is *larger than we want for changing the tone.*

  - *We'll need to use the **map()** math block to change the 0-1023 analogRead value into something more usable in this situation. Put the following block at the top of your program.*



Range of -5 to +5 means the pitch can go up or down 5 semitones



To modify the original tone – use the **math addition block** and add the value of **pitchShift**. Since this variable will be holding a value somewhere between -5 and +5, this will cause the initial tone value (in this case, 50) to be raised or lowered by a small amount.

Repeat this pattern for each of your **playSpeaker** blocks. Make sure that the left value in the addition block is the *original tone value* for that note.

You might think this is a bit tedious, and you'd be right. We're limited a bit in the block coding environment; as we learn more about text-based syntax we'll learn some strategies that could make the code here much more elegant. In the meantime, **right click and duplicate the green addition block**; that's about the best we can do here.



Once you've done that, test it out in the simulator and physical circuit before moving on to the next challenge.

With funding from Canada

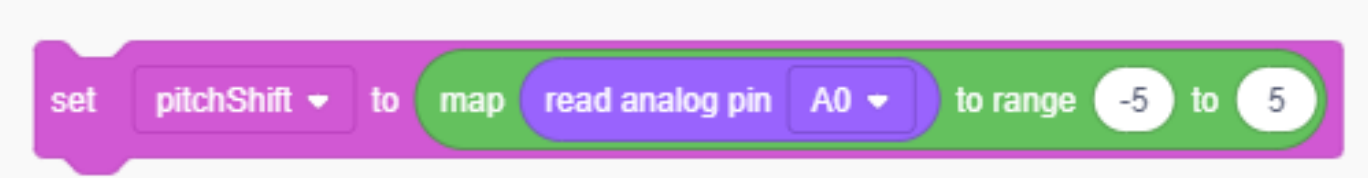
SIEC Saskatoon Industry Education Council

<SaskCode>

Challenge 2b – More Responsive Pitch Control

One drawback to the approach to pitch control we employed is that the variable **pitchShift** is only updated once, at the very start of the program. Once the notes begin to play, moving the potentiometer around will have no impact on the sounds until the whole song finishes. At that point, another analogRead() will take place and **pitchShift** will be updated, possibly starting the song in a new key if the potentiometer is at a different angle at that time.

Similar to above, we can make this more responsive, but with the constraints of the block environment, it is pretty crude. If we duplicate the block at *reads the potentiometer value:*

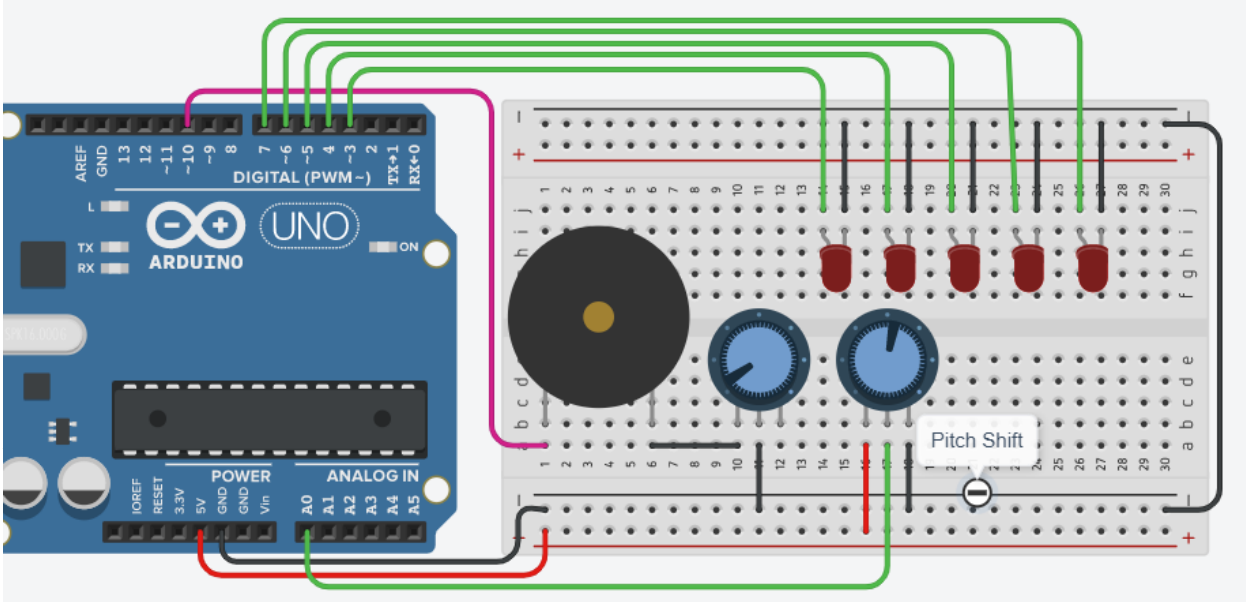


We can put a copy of this block before every **playSpeaker** block. This will allow us to update the **pitchShift** variable much more frequently, and changes to the potentiometer will be reflected after only waiting for one note to finish rather than the whole song.

Similar to what was mentioned before, we could build a much more elegant approach to accomplishing this task when working with syntax-based code.

Challenge Three – LED Visualization

For the third challenge, you'll add five LEDs and will turn one on for each of the five possible notes that are played in the Happy Birthday song. Start with your circuit:



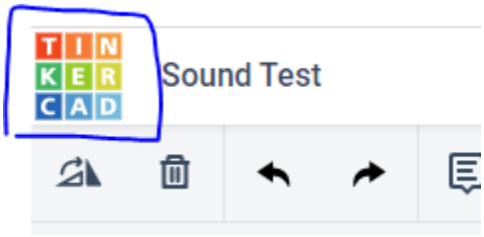
With funding from Canada
SIEC Saskatoon Industry Education Council
<SaskCode>

Note that in the circuit the leftmost LED is in the lowest digital pin (3) and the rightmost is in the highest digital pin being used (7).

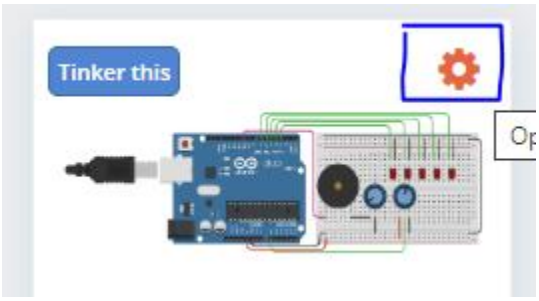
Stop – Duplicate your Project and Switch to Text-based

This last challenge would be even more tedious than the block-duplication we employed earlier, so we will look at **making functions** in text-coding mode. However, to prevent losing the ability to modify your tone blocks, let's make a duplicate of this project so we can preserve what you have up until this point:

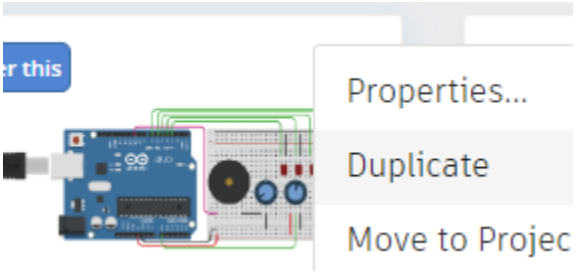
Click on the TinkerCAD logo in the top-left:



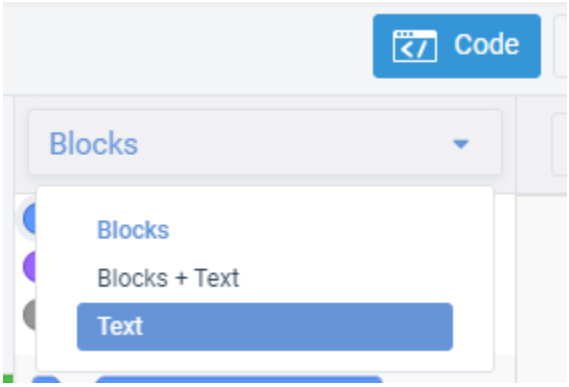
Find the project you were working on and look for the cog in the project's top-right corner.



Pick "Duplicate" from the options provided. This will open up a new copy of that project.



Finally, click on <Code> and change to **text** mode.



With funding from Canada
SIEC Saskatoon Industry Education Council
<SaskCode>

Functions

In this case we have 5 LEDs, each connected to a different digital pin. To have only one on at a time, any time we want to make a change we'll need to **turn four to LOW and one to HIGH.**

There are five possible combinations, and we'll need to pick one to do for each sound that is played! That *could be* a lot of coding, but it doesn't have to be. One approach is to build a function of our own – this like making your own command that you can ask to happen in code multiple times.

Add the following code to the bottom of your text editor. (It doesn't actually need to be at the bottom, but it can't be inside any sets of script brackets)!

```
void led1() {     //turn leftmost LED on only
   digitalWrite(3, HIGH);
   digitalWrite(4, LOW);
   digitalWrite(5, LOW);
   digitalWrite(6, LOW);
   digitalWrite(7, LOW);
}
```

This basically creates a new command called **led1()**, which we can call whenever we want. Five lines of code for the price of one! Once you have this in place, create four more – led2(), led3(), led4(), led5() – the only difference between them should be which pin gets set to HIGH (on). Follow the pattern left to right.

Also, make sure to add the following to the setup() function – since we didn't use any digitalWrite() blocks for the LED pins this wasn't added automatically for us:

```
void setup() {
   pinMode(3, OUTPUT);
   pinMode(4, OUTPUT);
   pinMode(5, OUTPUT);
   pinMode(6, OUTPUT);
   pinMode(7, OUTPUT);
   pinMode(A0, INPUT);       //should be there already
   pinMode(10, OUTPUT);   //should be there already
}
```

Now you simply need to activate the functions! You'll add a function call *before each* ***tone()*** *command.* You should be able to see in the code which note is going to be played, so pick the LED function that matches how high that note is:

```
void loop()
{
  pitchShift = map(analogRead(A0), 0, 1023, -5, 5);
  tone(10, 440 * pow(2.0, (constrain(int((50 + pitchShift)), 35
  led1();   //function for leftmost LED
```

Granted, there is a lot going on in the tone() command – mostly this is the conversion TinkerCAD is automatically doing to translate from the tone value to a frequency in hertz. See above how 50 is highlighted – that is the **tone value.** The function to call should match the following table:

Stanza One:

Tone Value	Function to Call
50	led1()
52	led2()
54	led3()
55	led4()
	led5()

Stanza Two:

Tone Value	Function to Call
50	led1()
52	led2()
55	led3()
57	led4()
	led5()

Stanza Three:

Tone Value	Function to Call
50, 52	led1()
54	led2()
55	led3()
59	led4()
62	led5()

Stanza Four:

Tone Value	Function to Call
	led1()
55	led2()
57	led3()
59	led4()
60	led5()

Once you've called the appropriate led function for each note being played, test it out in the simulator. If it works properly, add the components to your physical circuit, copy the new code to your Arduino, and try it out there.

Make sure to show me the completed physical circuit for this challenge!

Challenge Extra – Different Songs

As you likely saw, it takes quite a bit of work to programmatically compose or transcribe a song. And very likely you won't need or want "Happy Birthday" in too many different projects. One fun thing about sound with Arduino is that the community has had many people create and share their audio code.

Make sure your previous code is saved somewhere, then do an Internet search to see what is out there for "buzzer songs" or "arduino buzzer":



Load this code into the simulator / physical circuit to see how it sounds. You may want to incorporate some music or sound effects into future projects!

Note: It is very likely that someone else's code has a different **pin assignment.** Go through their code to see which digital pin their piezo buzzer was on. You can either re-wire your buzzer or change the code; do whichever is less work.